

Josh Williams

SQL Portfolio Addendum

Strong Query Examples with Story Problems, Explanations, and QA Use Cases

This addendum strengthens the SQL QA portfolio by adding three practical examples that demonstrate HAVING, EXCEPT, and IS NULL.

Portfolio Addendum Overview

Example	SQL Concept	Why It Strengthens the Portfolio
Candidate Skill Requirement Validation	HAVING	Validates that a candidate meets all required qualifications.
Facebook Page Engagement Validation	EXCEPT	Finds records that exist in one table but not another.
Tesla Assembly Bottleneck Investigation	IS NULL	Identifies incomplete production records and missing completion dates.

QA-SQL-013: Candidate Skill Requirement Validation

Primary SQL Concepts:

WHERE IN, GROUP BY, HAVING, COUNT(), ORDER BY

Story Problem:

Given a table of candidates and their skills, the business needs to find candidates best suited for an open Data Science job. Candidates must be proficient in Python, Tableau, and PostgreSQL. Write a query to list candidates who possess all of the required skills for the job. Sort the output by candidate ID in ascending order. Assumption: there are no duplicates in the candidates table.

SQL Query:

```
SELECT candidate_id
FROM candidates
WHERE skill IN ('Python', 'Tableau', 'PostgreSQL')
GROUP BY candidate_id
HAVING COUNT(skill) = 3
ORDER BY candidate_id;
```

Explanation:

The WHERE clause first limits the data to only the three required skills. GROUP BY candidate_id evaluates each candidate separately. HAVING COUNT(skill) = 3 keeps only candidates who have all three required skills. ORDER BY candidate_id returns the final list in ascending order.

Expected Result:

The result should include only candidate IDs for candidates who have Python, Tableau, and PostgreSQL.

QA Use Case:

This query validates candidate search filters, job qualification rules, applicant matching logic, and recruiting dashboard results.

Possible Defects Identified:

Candidates missing required skills appearing in results; qualified candidates being excluded; incorrect filter logic; missing skill records; qualification engine mismatches.

Skills Demonstrated:

HAVING, WHERE IN, GROUP BY, COUNT(), business rule validation, requirement verification.

QA-SQL-014: Facebook Page Engagement Validation

Primary SQL Concepts:

EXCEPT, Set Operations, ORDER BY

Story Problem:

Assume you are given two tables containing data about Facebook Pages and their respective likes. Write a query to return the IDs of the Facebook pages that have zero likes. The output should be sorted in ascending order based on the page IDs.

SQL Query:

```
SELECT page_id
FROM pages
EXCEPT
SELECT page_id
FROM page_likes
ORDER BY page_id;
```

Explanation:

The first SELECT returns every Facebook page. The second SELECT returns pages that have received at least one like. EXCEPT removes all liked pages from the complete page list, leaving only pages with zero likes. ORDER BY sorts the remaining page IDs in ascending order.

Expected Result:

The result should include only page IDs that exist in pages but do not exist in page_likes.

QA Use Case:

This query validates engagement reports, content dashboards, recommendation logic, and table relationship accuracy.

Possible Defects Identified:

Pages incorrectly shown as liked; missing like records; reporting discrepancies; data synchronization issues between page and like tables.

Skills Demonstrated:

EXCEPT, set operations, relationship validation, data quality testing, reporting verification.

QA-SQL-015: Tesla Assembly Bottleneck Investigation

Primary SQL Concepts:

IS NULL, Data Quality Validation

Story Problem:

Tesla is investigating production bottlenecks and needs to extract relevant production data. Write a query to determine which parts have begun the assembly process but are not yet finished. The parts_assembly table contains all parts currently in production, each at varying stages of the assembly process. An unfinished part is one that lacks a finish_date.

SQL Query:

```
SELECT
    part,
    assembly_step
FROM parts_assembly
WHERE finish_date IS NULL;
```

Explanation:

The query selects the part and current assembly_step from the production table. The WHERE finish_date IS NULL condition identifies parts that do not yet have a completion timestamp, meaning they are still unfinished.

Expected Result:

The result should return every part currently in production that has no finish_date.

QA Use Case:

This query validates manufacturing dashboards, work-in-progress reports, production status logic, and operational KPI reporting.

Possible Defects Identified:

Completed parts missing finish dates; unfinished parts incorrectly marked complete; production status mismatches; dashboard reporting inaccuracies; data entry errors.

Skills Demonstrated:

IS NULL, missing data validation, production workflow analysis, exception reporting, defect investigation.

How These Examples Improve the Main SQL Portfolio

Together, these examples add three important areas of SQL coverage: HAVING for grouped business rule validation, EXCEPT for set-based relationship checks, and IS NULL for missing data validation. These are valuable additions because they show practical QA thinking beyond simply writing queries.

Portfolio Gap Filled	Example That Covers It
HAVING / grouped filtering	Candidate Skill Requirement Validation
Set operations	Facebook Page Engagement Validation
NULL / missing data validation	Tesla Assembly Bottleneck Investigation
Business rule testing	All three examples
Defect investigation mindset	All three examples